Experiment No -11 (Group D)

Title:-

Implementation of Various Operations on a Binary Search Tree

Problem statement:-

Implement various operations on a Binary Search Tree, such as insertion, deletion, display, and search.

Pre-requisite:-

- Basic knowledge of **Python programming**.
- Understanding of **functions, classes, and objects**.
- Knowledge of basic **data structures and trees**.
- Understanding of **recursion**.
- Familiarity with **binary trees and searching techniques**.

Objective:-

- To understand the concept and structure of a **Binary Search Tree (BST)**.
- To implement **insertion** of elements into a BST.
- To implement **deletion** of elements from a BST.
- To implement **search** operations in a BST.

Software and Hardware requirements:-

Software :-

Operating System recommended:- 64-bit Open source Linux or Windows or macOS

Programming tools recommended: - Python 3.x

Programming IDE/Editor: IDLE, VS Code, PyCharm, or any Python-supported editor

Hardware :-

Processor: Intel Core i3 or equivalent and above

RAM: Minimum 2 GB

Storage: Minimum 100 MB free space

Keyboard and Monitor: Required for entering input and viewing output.

Outcome:-

After completing this experiment, the student will be able to:

- Explain the structure and properties of a **Binary Search Tree**.
- Create and maintain a BST using Python.
- Perform **insertion, deletion, and searching** operations.
- Display BST elements using **inorder, preorder, and postorder traversal**.

Theory:-

A **Binary Search Tree (BST)** is a binary tree in which each node follows a specific ordering property:

- All elements in the **left subtree** are smaller than the root node.
- All elements in the **right subtree** are greater than the root node.
- Each left and right subtree is also a Binary Search Tree.

Consider a binary tree T. T is a binary search tree, that is, the value of the data elements in every node N in T less than to the data elements in every node in left subtree and is greater than or equal to the data elements in every node in right subtree.

60, 100, 55, 67, 107, 45, 57

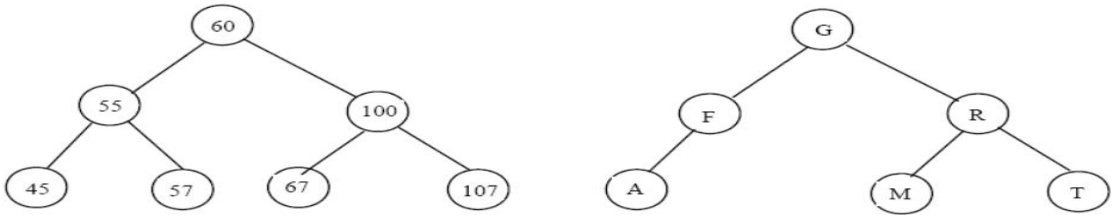


Figure 2. Binary Search Tree (BST)

Linked Representation of Binary Search Tree

A binary search tree can be represented using a set of linked nodes. Each node contains a value and two links named left and right that reference the left child and right child, respectively

Pointer to Left Subtree	Data	Pointer to Right Subtree

Figure 3. Node

The variable *root* refers to the root node of the tree. If the tree is *empty*, root is NULL.

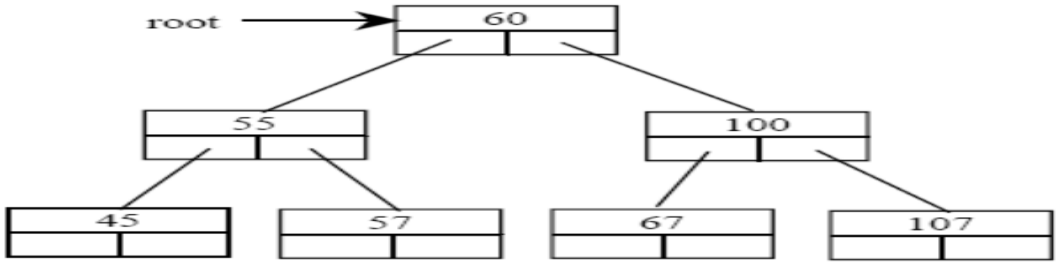


Figure 4. Linked Representation of Binary Search Tree

Algorithms for BST Operations

1. Algorithm for Insertion

Algorithm: INSERT(root, key)

1. Start.
2. If root is None, create a new node with key and return it.
3. If key < root.key:
    - Recursively insert key into the left subtree.
    - Set root.left to the returned node.
4. Else if key > root.key:
    - Recursively insert key into the right subtree.
    - Set root.right to the returned node.
5. Return root.
6. Stop.

2. Algorithm for Search

Algorithm: SEARCH(root, key)

1. Start.
2. If root is None, return False because the element is not present.
3. If root.key == key, return True.
4. If key < root.key:
    - Search recursively in the left subtree.
5. Otherwise:
    - Search recursively in the right subtree.
6. Return the result.
7. Stop.

3. Algorithm for Deletion

Algorithm: DELETE(root, key)

1. Start.
2. If root is None, return root.
3. If key < root.key:
    - Recursively delete key from the left subtree.
4. Else if key > root.key:
    - Recursively delete key from the right subtree.
5. Otherwise, the node to be deleted is found.
6. Check the number of children:
    - No left child: Return the right child.
    - No right child: Return the left child.
    - Two children:
        1. Find the minimum-valued node in the right subtree.
        2. Copy its value into the current node.
        3. Delete the minimum-valued node from the right subtree.
7. Return root.
8. Stop.

4. Algorithm for Finding Minimum Value

This is used during deletion when the node has two children.

```
MIN_VALUE(root)
  current = root

  WHILE current.left is not NULL
    current = current.left

  RETURN current
```

The minimum value is always found by continuously moving to the leftmost node of the subtree.

Tree traversals:-

Note:- These tree traversal are any kind of binary tree including Binary Search Tree.

Traversing a tree means visiting all nodes which are present in that tree.

There are 3 types of traversal

1. Inorder traversal
2. Preorder traversal
3. Postorder traversal

These three traversal can be done with or without recursion.

Recursive algorithm for inorder traversals-:-

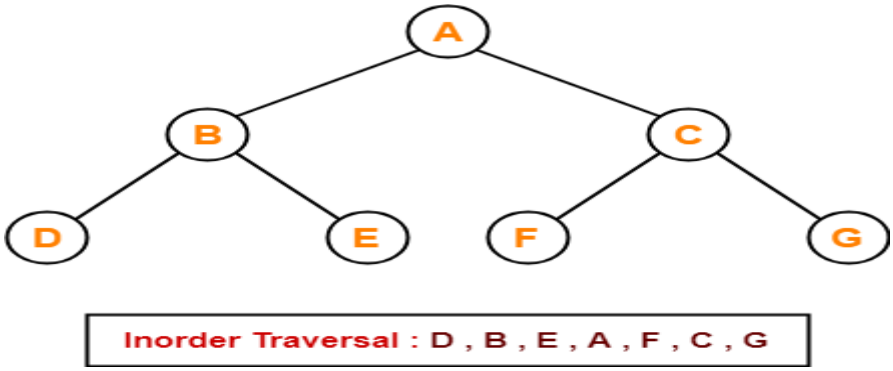
In this traversal method first process left element, then root element and then the right element.

Procedure:- Left → Root → Right

Step 1- Recursively traverse the left subtree

Step 2- Visit root node

Step 3- Recursively traverse right



Recursive algorithm for preorder traversals -

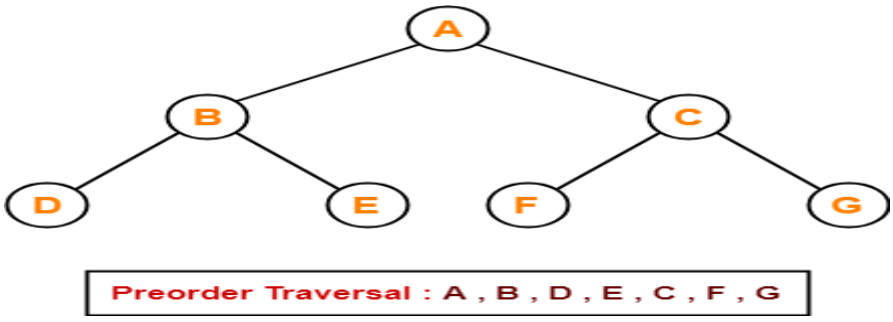
In pre-order traversal, it visits the root node first, then the left subtree, and lastly right subtree.

Algorithm: **Root → Left → Right**

Step 1- Visit root node

Step 2- Recursively traverse the left subtree

Step 3- Recursively traverse right subtree



Recursive algorithm for postorder traversals -

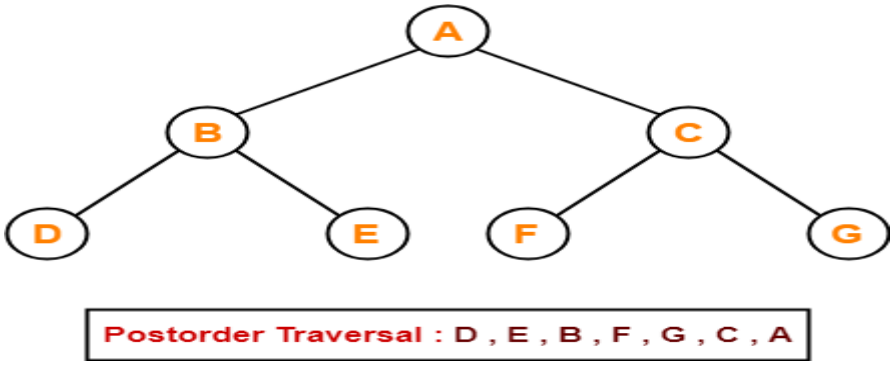
It visits the left subtree first in post-order traversal, then the right subtree, and finally the root node.

Algorithm: Left → Right → Root

Step 1- Recursively traverse the left subtree

Step 2- Recursively traverse right subtree

Step 3- Visit root node



Conclusion:-

Thus, the various operations on a **Binary Search Tree**, including **insertion, deletion, searching, and traversal/display**, were successfully implemented in Python. The experiment demonstrates how BSTs organize data efficiently and how different traversal techniques can be used to access and display the stored elements.

Viva Questions:-

1. What is a Binary Tree?
2. What is a Binary Search Tree (BST)?
3. What are the main operations performed on a BST?
4. What is the root node?
5. What is a leaf node?
6. What is a parent node?
7. What is a subtree?
8. What is the difference between a binary tree and a BST?
9. What is the time complexity of searching in a BST?
10. What are the different tree traversal techniques?
11. What is inorder traversal?
12. What is preorder traversal?
13. What is postorder traversal?
14. What is the height of a BST?
15. What is a degenerate BST?